

Detection Engineering & Alert Triage Report

1. Executive Summary

I updated my existing SOC lab to better simulate the day-to-day activities of a Tier 1 SOC Analyst, with a focus on detecting, investigating, and triaging security events. The project involved validating existing log sources, developing five custom Wazuh detection rules mapped to MITRE ATT&CK techniques, and testing those rules through simulated and manually generated activities. I also created a triage workflow to provide a consistent process for reviewing alerts, determining their severity, and deciding whether they should be escalated or closed. The result is a more practical SOC environment where security events can be visualized, detected, investigated, and documented using a structured workflow.

2. Scope & Objective

The main objective of this project was to improve my existing SIEM/SOC lab so that it could more closely represent the activities performed by a Tier 1 SOC Analyst.

The project focused on:

- Validating that the required log sources were actively sending useful telemetry to Wazuh.
- Identifying and fixing gaps in the Linux audit logging configuration.
- Creating custom detection rules for selected MITRE ATT&CK techniques.
- Testing the detection rules using Atomic Red Team and manually simulated activities.
- Observing alert behavior and tuning rules that generated excessive noise.
- Creating a triage workflow that could be followed when investigating alerts.
- Applying the workflow to nine alerts and documenting their final dispositions.

The overall goal was to make malicious or suspicious activity easier to detect and visualize while providing a repeatable process for analysing the alerts generated by the lab.

3. Methodology

3.1 Phase 1 — Log Source Validation

The first phase focused on validating the log sources already configured in the SOC lab. Rather than immediately creating new detection rules, I manually generated activities and checked whether the resulting events were reaching Wazuh correctly.

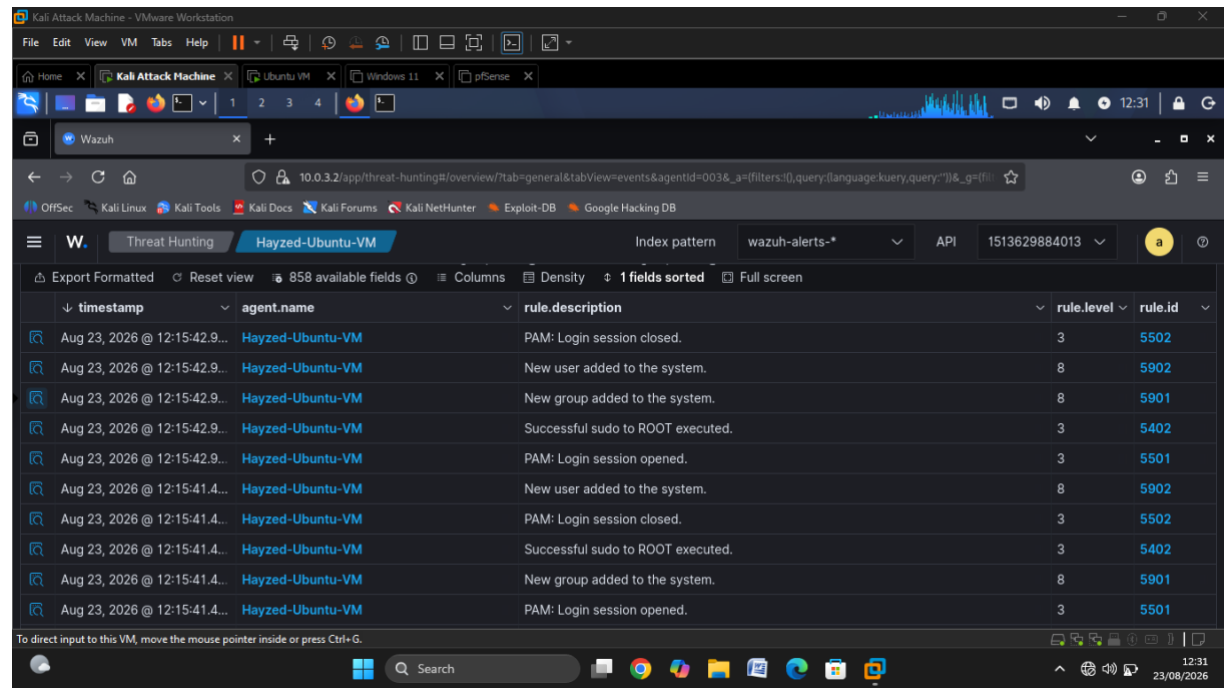
During this process, I discovered that Auditd was not installed on the Ubuntu endpoint. Since several of the detection rules planned for the project depended on Auditd telemetry, this had to be resolved before continuing.

I installed Auditd on the Ubuntu endpoint, configured the required audit rules, and added the appropriate Auditd log collection configuration to the Wazuh agent configuration file. I then manually generated an authentication-related activity and confirmed that the resulting Auditd event was successfully received and displayed in Wazuh.

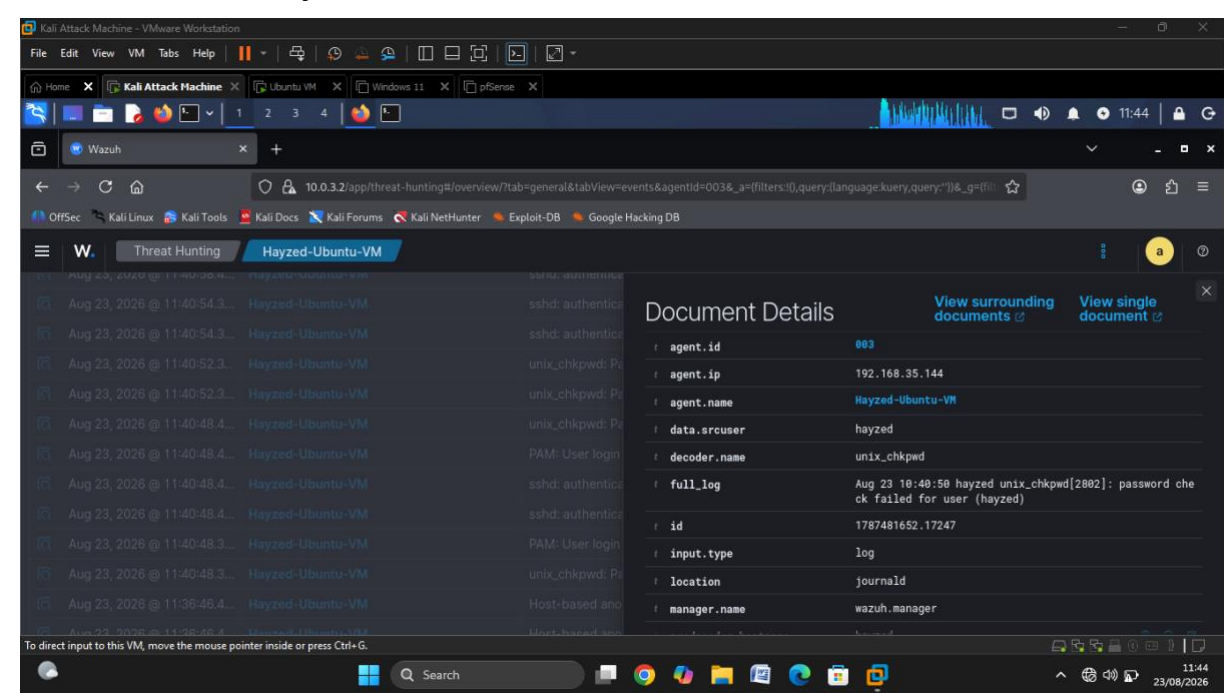
The validation also confirmed that Linux system logs, Windows Sysmon/process telemetry, and firewall/network logs were being populated in Wazuh.

Evidence:

- Linux authentication and system event ingestion.



- Detailed Linux journald authentication event.



- Firewall/network gateway log ingestion.

The screenshot shows the Wazuh Discover interface in a Kali Linux VM. The search bar contains 'wazuh-alerts-*'. The left sidebar shows 'Selected fields' with '_source' and 'Available fields' including '_index', 'agent.id', 'agent.ip', 'agent.name', 'data.arch', and 'data.command'. The main panel displays a log entry with the following details:

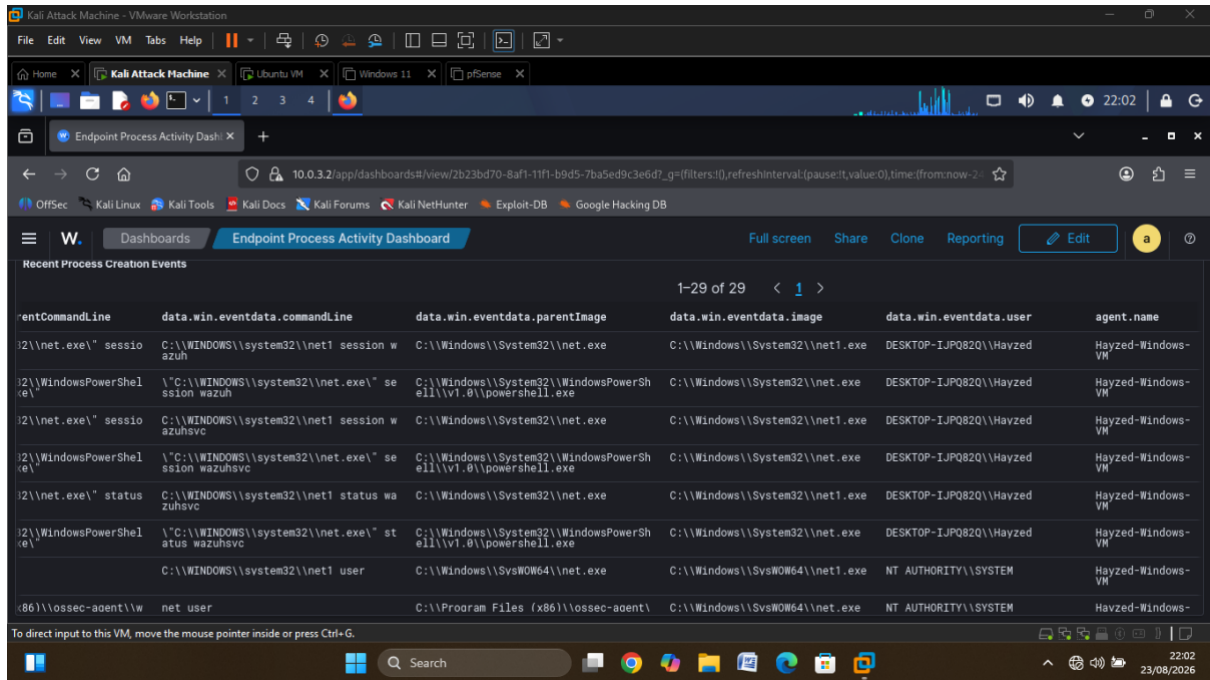
field	value
decoder.name	kernel
full_log	Aug 23 19:31:11 kernel: plog0: promiscuous mode enabled
id	1787518383.187600
input.type	log
location	10.0.3.1
manager.name	wazuh.manager
predecoder.hostname	kernel:
predecoder.program_name	kernel

- Syslog, SSH authentication activity and brute-force correlation

The screenshot shows the Wazuh Threat Hunting interface. The search bar contains 'wazuh-alerts-*'. The left sidebar shows 'Selected fields' with '_source' and 'Available fields' including '_index', 'agent.id', 'agent.ip', 'agent.name', 'data.arch', and 'data.command'. The main panel displays a list of security events with the following columns: timestamp, agent.name, rule.description, rule.level, and rule.id.

timestamp	agent.name	rule.description	rule.level	rule.id
Aug 23, 2026 @ 12:33:06.3...	Hayzed-Ubuntu-VM	syslog: User missed the password more than one time	10	2502
Aug 23, 2026 @ 12:33:06.3...	Hayzed-Ubuntu-VM	sshd: Attempt to login using a non-existent user	5	5710
Aug 23, 2026 @ 12:33:06.2...	Hayzed-Ubuntu-VM	syslog: User missed the password more than one time	10	2502
Aug 23, 2026 @ 12:33:06.2...	Hayzed-Ubuntu-VM	sshd: Attempt to login using a non-existent user	5	5710
Aug 23, 2026 @ 12:32:56.2...	Hayzed-Ubuntu-VM	sshd: Attempt to login using a non-existent user	5	5710
Aug 23, 2026 @ 12:32:56.2...	Hayzed-Ubuntu-VM	Possible SSH Brute Force Attack: 5+ failed logins from the same IP within 5 minutes.	12	100300
Aug 23, 2026 @ 12:32:50.2...	Hayzed-Ubuntu-VM	sshd: Attempt to login using a non-existent user	5	5710
Aug 23, 2026 @ 12:32:50.2...	Hayzed-Ubuntu-VM	sshd: Attempt to login using a non-existent user	5	5710
Aug 23, 2026 @ 12:32:48.2...	Hayzed-Ubuntu-VM	PAM: User login failed.	5	5503
Aug 23, 2026 @ 12:32:48.2...	Hayzed-Ubuntu-VM	PAM: User login failed.	5	5503

- Windows Sysmon process creation telemetry displayed in the Endpoint Process Activity Dashboard.



These checks established that the major log sources required for the detection engineering phase were operational.

3.2 Phase 2 — Detection Rule Development

The second phase focused on developing five custom detection rules based on MITRE ATT&CK techniques relevant to the lab.

I reviewed the MITRE ATT&CK framework and selected techniques covering persistence, execution, and credential access. The selected techniques were:

Rule ID	MITRE ATT&CK Technique	Technique	Wazuh Level
100402	T1053.003	Scheduled Task/Job: Cron	8
100405	T1543.002	Systemd Service	9
100407	T1059.004	Command and Scripting Interpreter: Unix Shell	10
100409	T1003.008	OS Credential Dumping	12
100410	T1552.001	Unsecured Credentials	10

The rules were created in Wazuh's `local_rules.xml` and were designed around the telemetry available from the Linux endpoint.

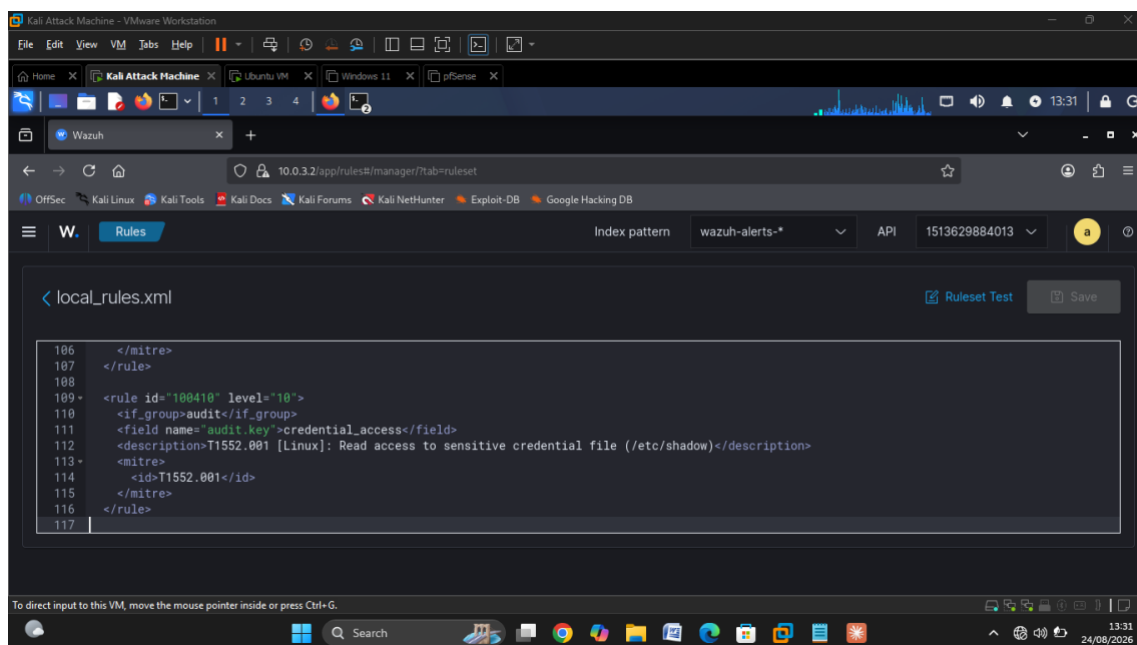
For the Scheduled Job and Systemd Service techniques, I used Wazuh File Integrity Monitoring to monitor specific directories and file paths associated with cron jobs and Systemd service units.

For the Auditd-based techniques, I created Auditd rules to monitor specific activities before creating the corresponding Wazuh detection logic. This was necessary because the Wazuh rule could only detect the activity if Auditd was first generating the appropriate telemetry.

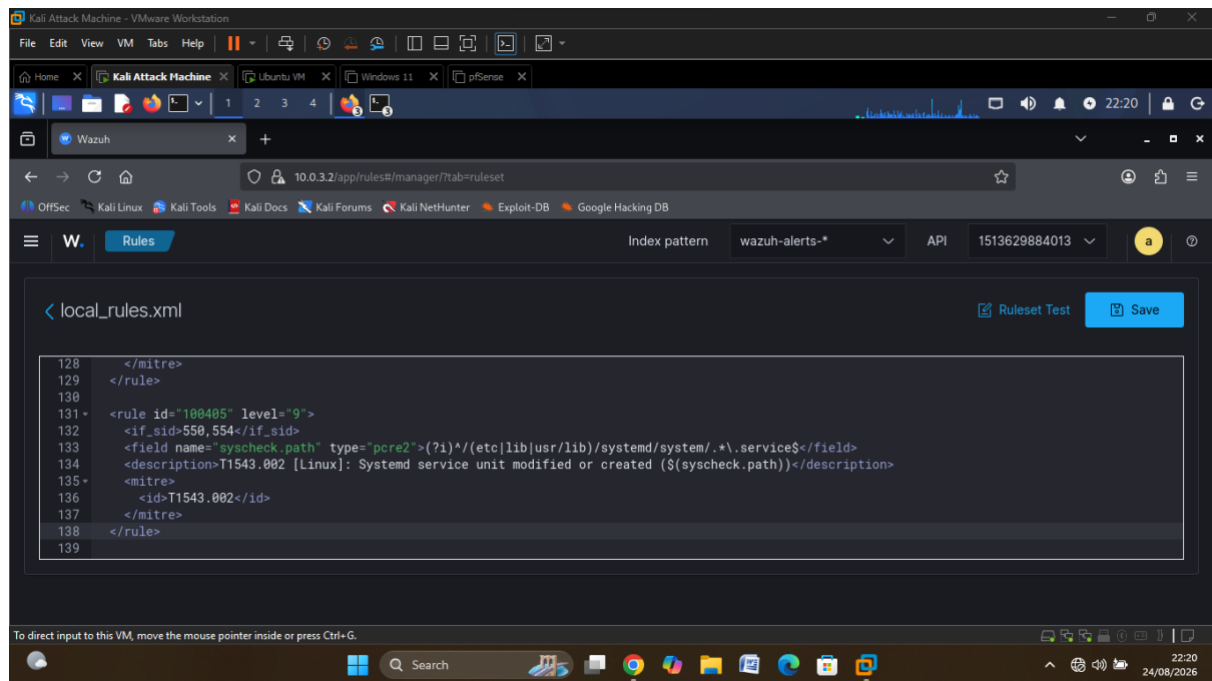
Each custom rule was also mapped to its corresponding MITRE ATT&CK technique.

Evidence:

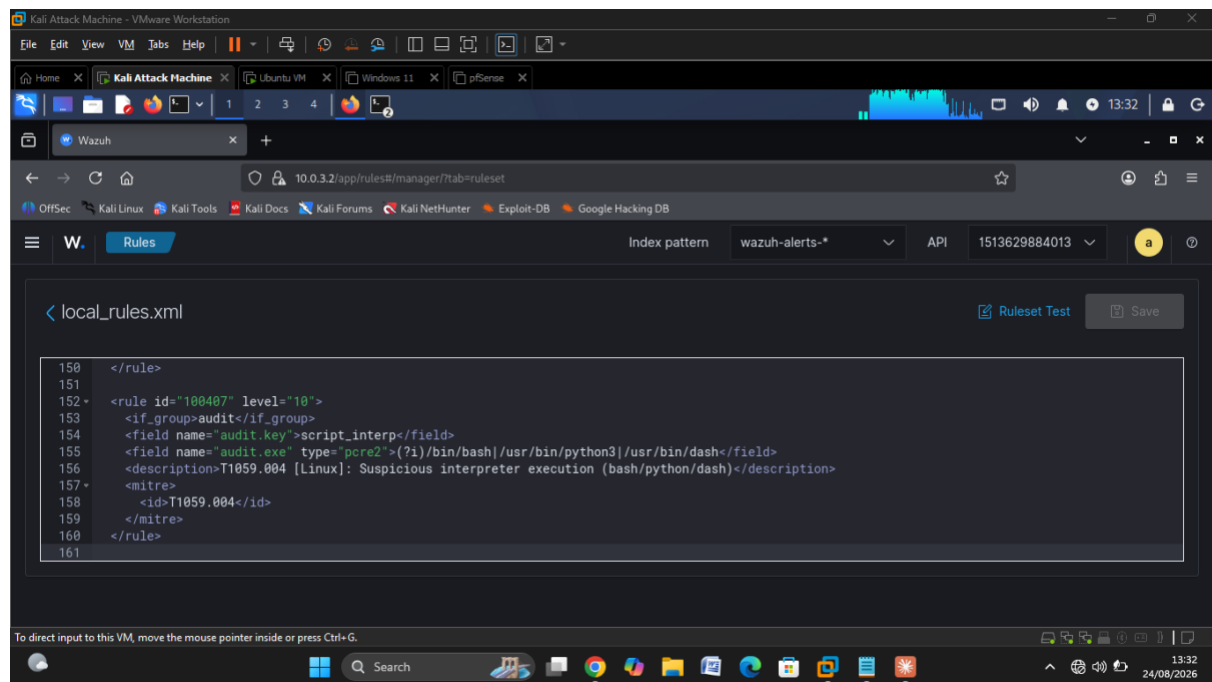
- Rule 100410: T1552.001.



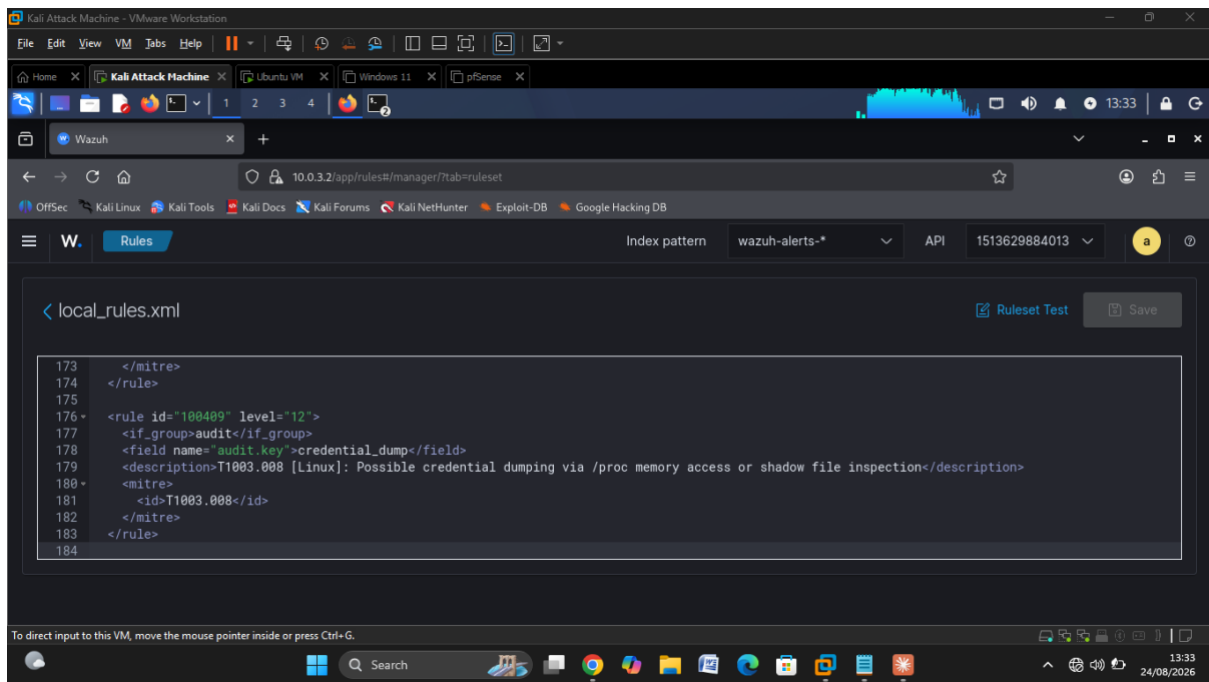
- Rule 100405: T1543.002.



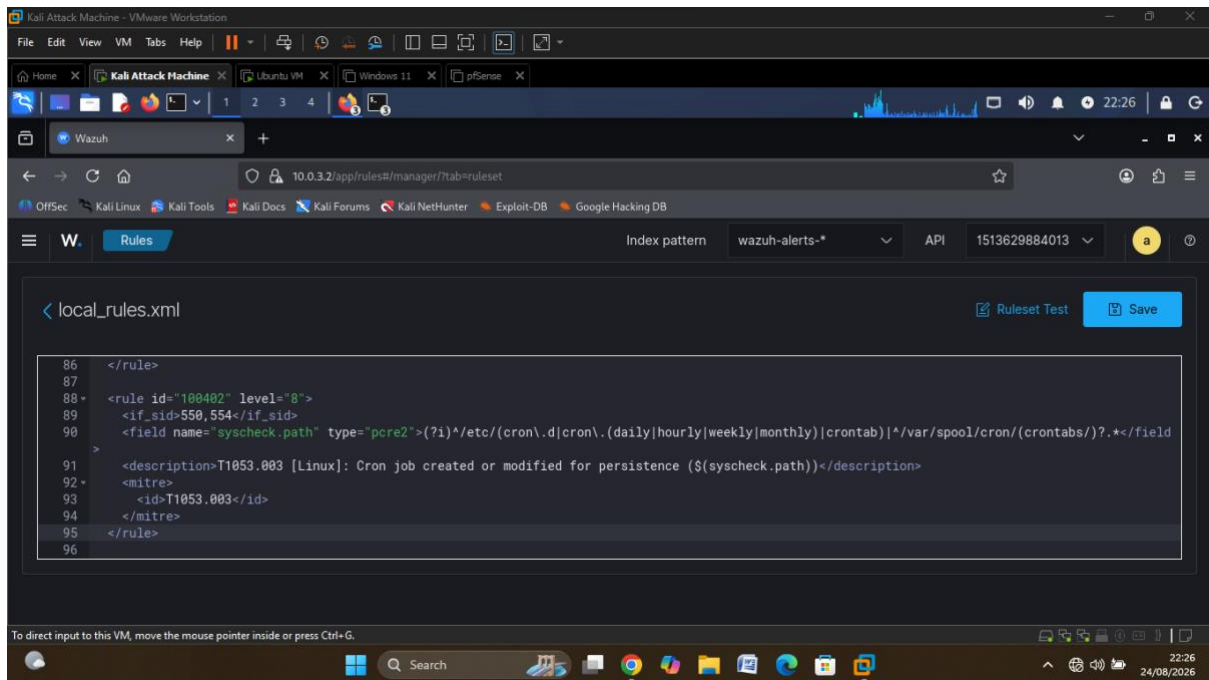
- Rule 100407: T1059.004.



- Rule 100409: T1003.008.



- Rule 100402: T1053.003.



3.3 Phase 3 — Attack Simulation & Detection Results

The third phase focused on testing whether the detection rules actually worked when the relevant activities occurred.

I used Atomic Red Team to simulate attack techniques against the Linux endpoint where applicable. However, Atomic Red Team did not provide an exact test for every detection scenario required by the project, so some activities were manually simulated within the isolated lab environment.

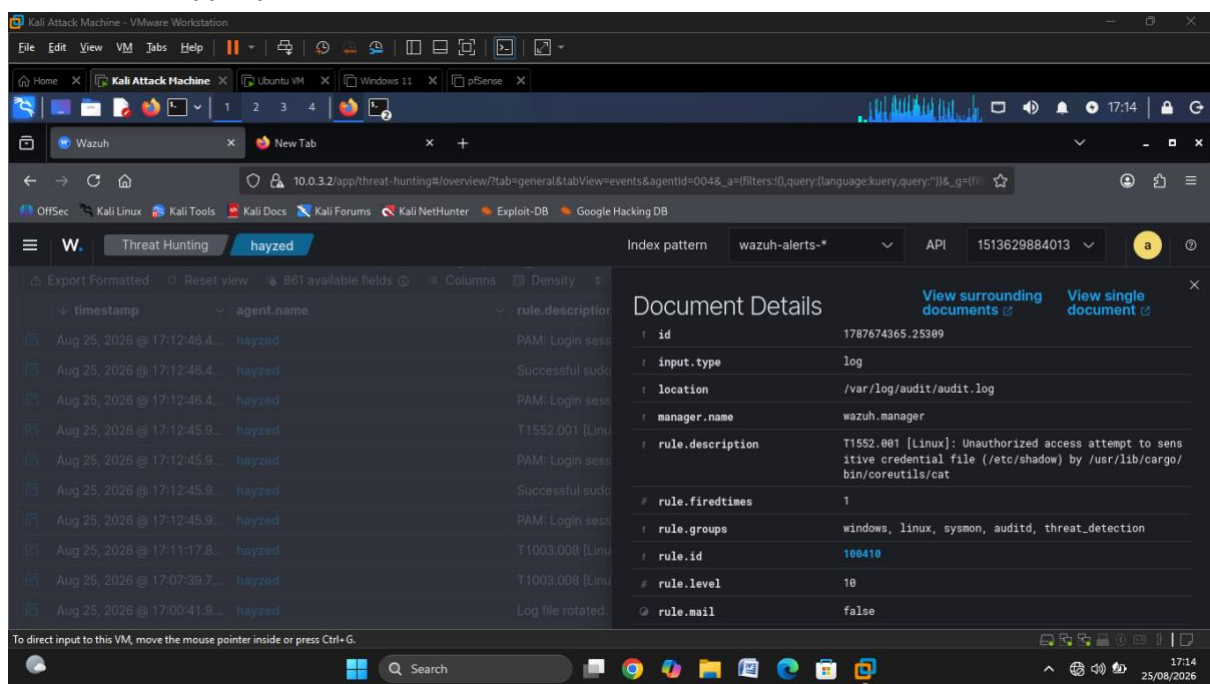
The five custom rules were successfully triggered during testing.

Rule 100410 generated a Level 10 alert when access to `/etc/shadow` was detected. Rule 100409 generated a Level 12 alert when Auditd telemetry matched the credential-dumping detection criteria. Rule 100402 detected the creation of a cron file under `/etc/cron.d/`, while Rule 100405 detected the creation of a Systemd service file. Rule 100407 also detected interactive Bash execution through Auditd.

The resulting Wazuh alerts provided evidence that the detection logic was not only configured but was actively generating alerts from relevant telemetry.

Evidence:

- Rule 100410 detection.



The screenshot shows a Kali Linux VM with a Wazuh alert interface. The main table displays a list of alerts, and a 'Document Details' sidebar provides information for rule 100410.

timestamp	agent.name	rule.description
Aug 25, 2026 @ 17:12:46.4...	hayzed	PAM: Login sess
Aug 25, 2026 @ 17:12:48.4...	hayzed	Successful sudo
Aug 25, 2026 @ 17:12:46.4...	hayzed	PAM: Login sess
Aug 25, 2026 @ 17:12:45.9...	hayzed	T1552.001 [Linux]
Aug 25, 2026 @ 17:12:45.9...	hayzed	PAM: Login sess
Aug 25, 2026 @ 17:12:45.9...	hayzed	Successful sudo
Aug 25, 2026 @ 17:12:45.9...	hayzed	PAM: Login sess
Aug 25, 2026 @ 17:11:17.8...	hayzed	T1003.008 [Linux]
Aug 25, 2026 @ 17:07:39.7...	hayzed	T1003.008 [Linux]
Aug 25, 2026 @ 17:00:41.9...	hayzed	Log file rotated.

Document Details	
id	1787674365.25309
input.type	log
location	/var/log/audit/audit.log
manager.name	wazuh.manager
rule.description	T1552.001 [Linux]: Unauthorized access attempt to sensitive credential file (/etc/shadow) by /usr/lib/cargo/bin/coreutils/cat
rule.firedtimes	1
rule.groups	windows, linux, sysmon, auditd, threat_detection
rule.id	100410
rule.level	10
rule.mail	false

- Rule 100409 detection.

The screenshot shows the Wazuh Threat Hunting interface. The main table lists events with columns for timestamp, agent name, and rule description. The 'Document Details' panel on the right shows the details for Rule 100409.

timestamp	agent.name	rule.description
Aug 25, 2026 @ 17:07:39.7...	hayzed	T1003.008 [Linux]
Aug 25, 2026 @ 17:00:41.9...	hayzed	Log file rotated.
Aug 25, 2026 @ 16:59:51.7...	hayzed	Auditd: SELinux
Aug 25, 2026 @ 16:59:49.4...	hayzed	PAM: Login sess
Aug 25, 2026 @ 16:59:49.3...	hayzed	sshd: authentic
Aug 25, 2026 @ 16:59:49.3...	hayzed	PAM: Login sess
Aug 25, 2026 @ 16:59:49.3...	hayzed	T1552.001 [Linux]
Aug 25, 2026 @ 16:59:49.3...	hayzed	T1552.001 [Linux]
Aug 25, 2026 @ 16:59:49.3...	hayzed	PAM: Login sess
Aug 25, 2026 @ 16:59:49.3...	hayzed	PAM: Login sess

Document Details

id	1787674859.21184
input.type	log
location	/var/log/audit/audit.log
manager.name	wazuh.manager
rule.description	T1003.008 [Linux]: Possible credential dumping via /proc memory access or shadow file inspection
rule.firedtimes	1
rule.groups	windows, linux, sysmon, auditd, threat_detection
rule.id	100409
rule.level	12

- Rule 100402 detection.

The screenshot shows the Wazuh Threat Hunting interface. The main table lists events with columns for timestamp, agent name, and rule description. The 'Document Details' panel on the right shows the details for Rule 100402.

timestamp	agent.name	rule.description
Aug 26, 2026 @ 00:00:36.7...	hayzed	PAM: Login sess
Aug 26, 2026 @ 00:00:36.7...	hayzed	Successful sudo
Aug 26, 2026 @ 00:00:36.2...	hayzed	T1059.004 [Linux]
Aug 26, 2026 @ 00:00:36.2...	hayzed	PAM: Login sess
Aug 26, 2026 @ 00:00:36.2...	hayzed	PAM: Login sess
Aug 26, 2026 @ 00:00:36.2...	hayzed	Successful sudo
Aug 26, 2026 @ 00:00:35.8...	hayzed	T1053.003 [Linux]
Aug 25, 2026 @ 23:55:32.6...	hayzed	PAM: Login sess
Aug 25, 2026 @ 23:55:32.6...	hayzed	Successful sudo
Aug 25, 2026 @ 23:55:32.6...	hayzed	PAM: Login sess
Aug 25, 2026 @ 23:55:32.0...	hayzed	T1059.004 [Linux]
Aug 25, 2026 @ 23:55:32.0...	hayzed	Successful sudo

Document Details

full_log	File /etc/cron.d/attacksim_final added Mode: realtime
id	1787698835.371033
input.type	log
location	syscheck
manager.name	wazuh.manager
rule.description	T1053.003 [Linux]: Cron job created or modified for persistence ()
rule.firedtimes	1
rule.groups	windows, linux, sysmon, auditd, threat_detection
rule.id	100402
rule.level	8

- Rule 100407 detection.

The screenshot shows the Wazuh Threat Hunting interface. The left sidebar displays a histogram of alerts over time. The main table lists alerts with columns for timestamp, agent name, and rule description. The right pane shows 'Document Details' for a specific alert.

timestamp	agent.name	rule.description
Aug 24, 2026 @ 20:56:49.171	hayzed	T1059.004 [Linux]: Interactive script interpreter execution on terminal (pts0) by user (/usr/bin/bash)
Aug 25, 2026 @ 20:51:58.5...	hayzed	ptSense: Firewa...
Aug 25, 2026 @ 20:51:58.5...	hayzed	ptSense: Firewa...
Aug 25, 2026 @ 20:51:58.5...	hayzed	ptSense: Firewa...

Document Details:

- es:3 comm="bash" exe="/usr/bin/bash" subj=unconfined k...
- id: 1787687800.203443
- input.type: log
- location: /var/log/audit/audit.log
- manager.name: wazuh.manager
- rule.description: T1059.004 [Linux]: Interactive script interpreter execution on terminal (pts0) by user (/usr/bin/bash)
- rule.firedtimes: 1
- rule.groups: windows, linux, sysmon, auditd, threat_detection
- rule.id: 100407
- rule.level: 10

- Rule 100405 detection.

The screenshot shows the Wazuh Threat Hunting interface. The left sidebar displays a histogram of alerts over time. The main table lists alerts with columns for timestamp, agent name, and rule description. The right pane shows 'Document Details' for a specific alert.

timestamp	agent.name	rule.description
Aug 26, 2026 @ 00:04:42.9...	hayzed	Successful sudo
Aug 26, 2026 @ 00:04:42.9...	hayzed	PAM: Login sess
Aug 26, 2026 @ 00:04:42.9...	hayzed	PAM: Login sess
Aug 26, 2026 @ 00:04:42.9...	hayzed	PAM: Login sess
Aug 26, 2026 @ 00:04:42.3...	hayzed	Successful sudo
Aug 26, 2026 @ 00:04:42.3...	hayzed	PAM: Login sess
Aug 26, 2026 @ 00:04:42.3...	hayzed	PAM: Login sess
Aug 26, 2026 @ 00:04:42.0...	hayzed	T1543.002 [Linu
Aug 26, 2026 @ 00:00:36.7...	hayzed	PAM: Login sess
Aug 26, 2026 @ 00:00:36.7...	hayzed	Successful sudo
Aug 26, 2026 @ 00:00:36.7...	hayzed	PAM: Login sess

Document Details:

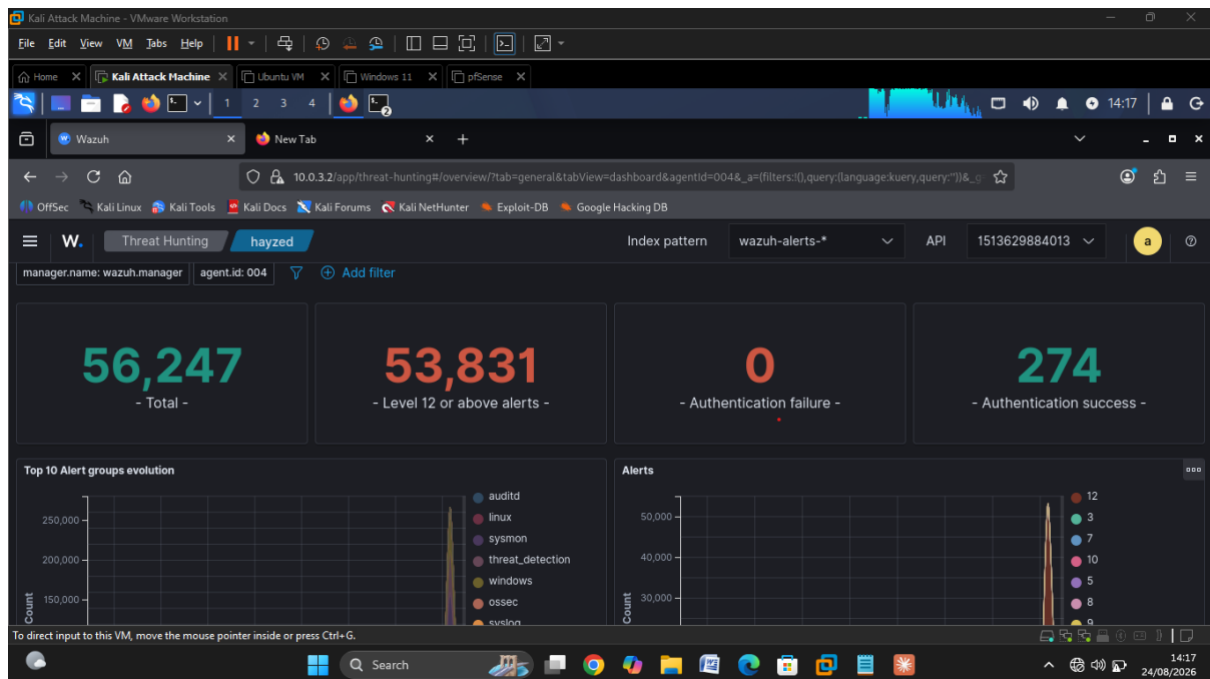
- Full_log: File /etc/systemd/system/atomic_test.service added Mode: realtime
- id: 1787699082.376709
- input.type: log
- location: syscheck
- manager.name: wazuh.manager
- rule.description: T1543.002 [Linux]: Systemd service unit modified or created ()
- rule.firedtimes: 1
- rule.groups: windows, linux, sysmon, auditd, threat_detection
- rule.id: 100405
- rule.level: 9

During testing, I also observed significant alert noise from some Auditd-based rules. In particular, Rule 100409 generated a very large number of Level 12 alerts within a short period. The initial telemetry showed more than 53,000 Level 12-or-higher alerts out of 56,247 total alerts during the observed period.

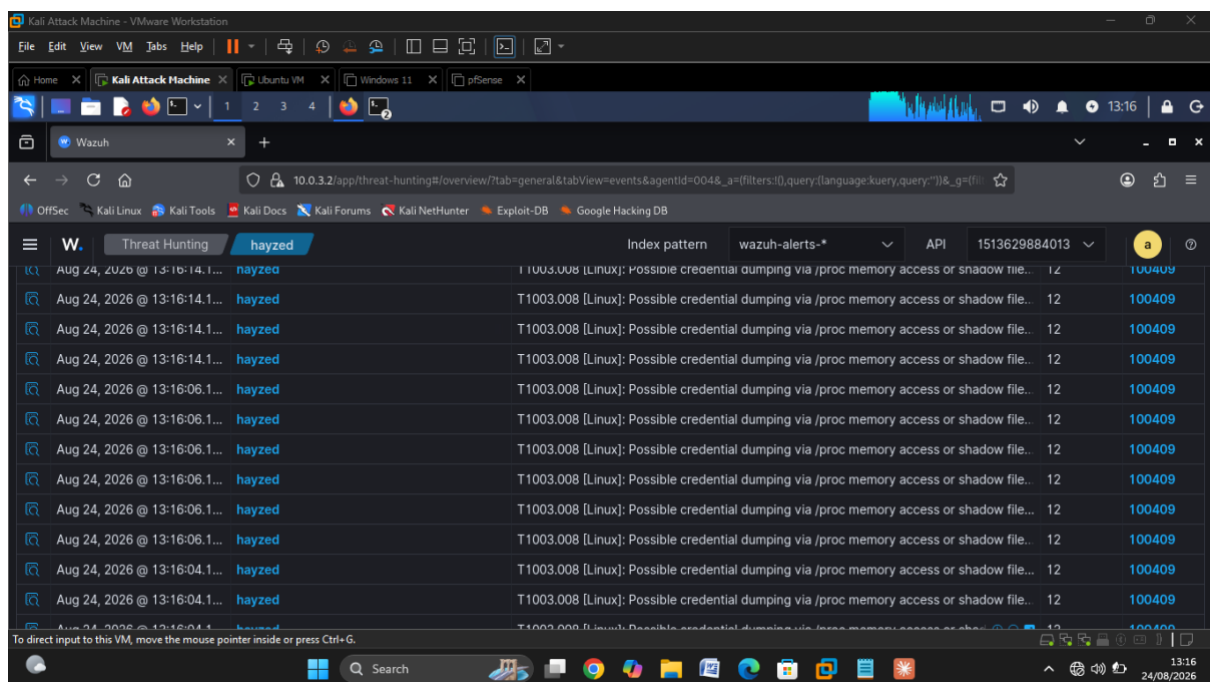
This demonstrated that although the detection logic was working, the initial scope was too broad for practical SOC use.

Evidence:

- Overall alert volume and alert spike.



- Repeated Rule 100409 alerts demonstrating alert noise



3.4 Detection Tuning

After identifying the excessive alert volume, I tuned the affected detection rules rather than leaving them in their original state.

For Rule 100407, I added filtering based on the terminal field so that the rule focused on interactive terminal sessions such as pts/X and ttyX. This reduced alerts generated by background or automated processes.

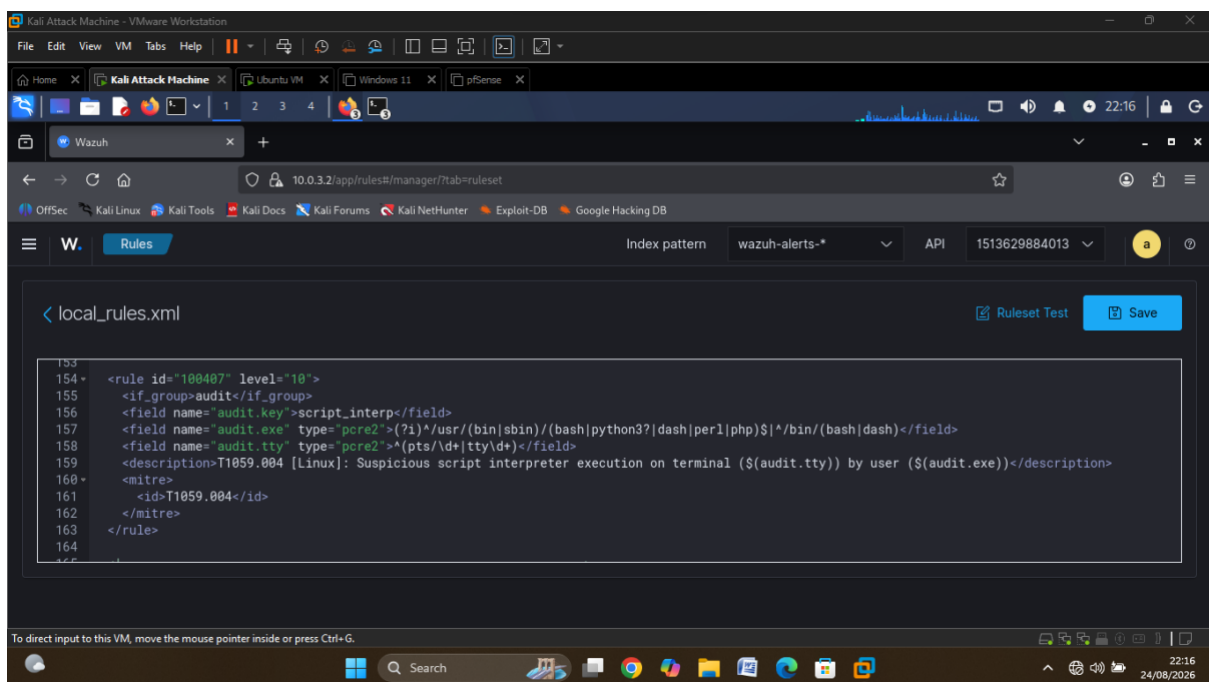
For Rule 100410, I added exclusions for known legitimate administrative and system processes, including utilities such as sudo, su, passwd, sshd, cron, and other trusted processes. This allowed the rule to continue monitoring sensitive credential access while reducing known benign activity.

For Rule 100409, I added an exception for specific debugging binaries that were contributing to unnecessary alerts.

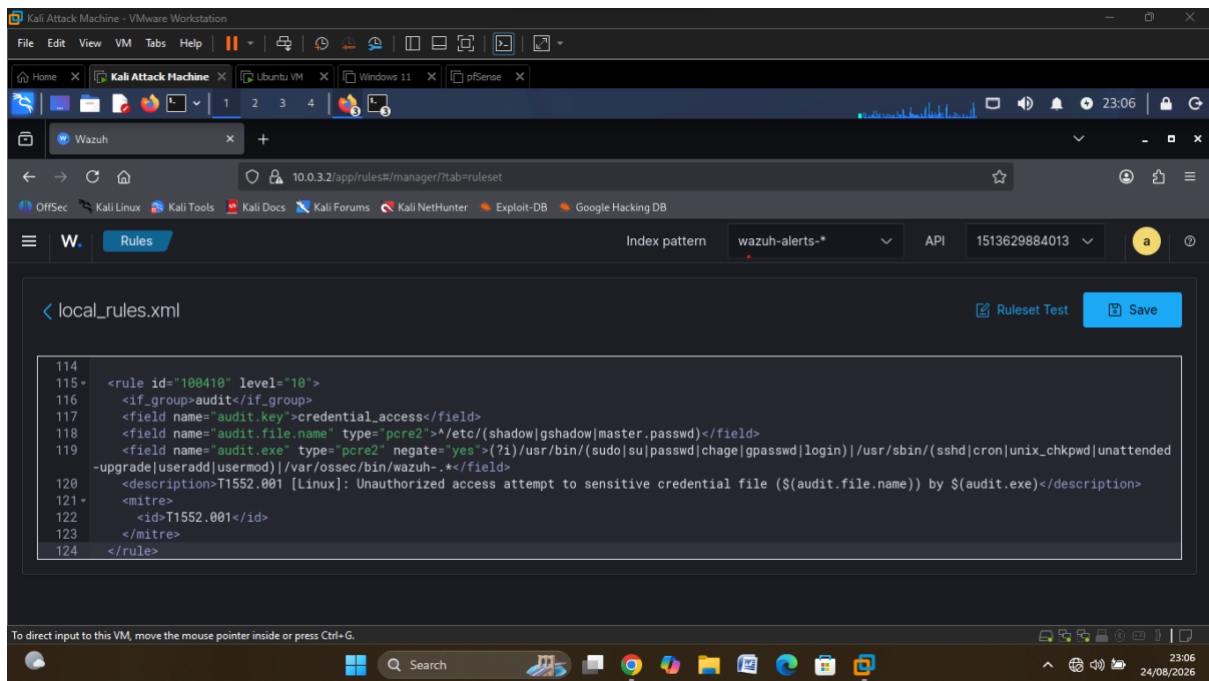
The tuning process demonstrated the difference between simply creating a detection rule and developing a detection that is more practical for SOC operations.

Evidence:

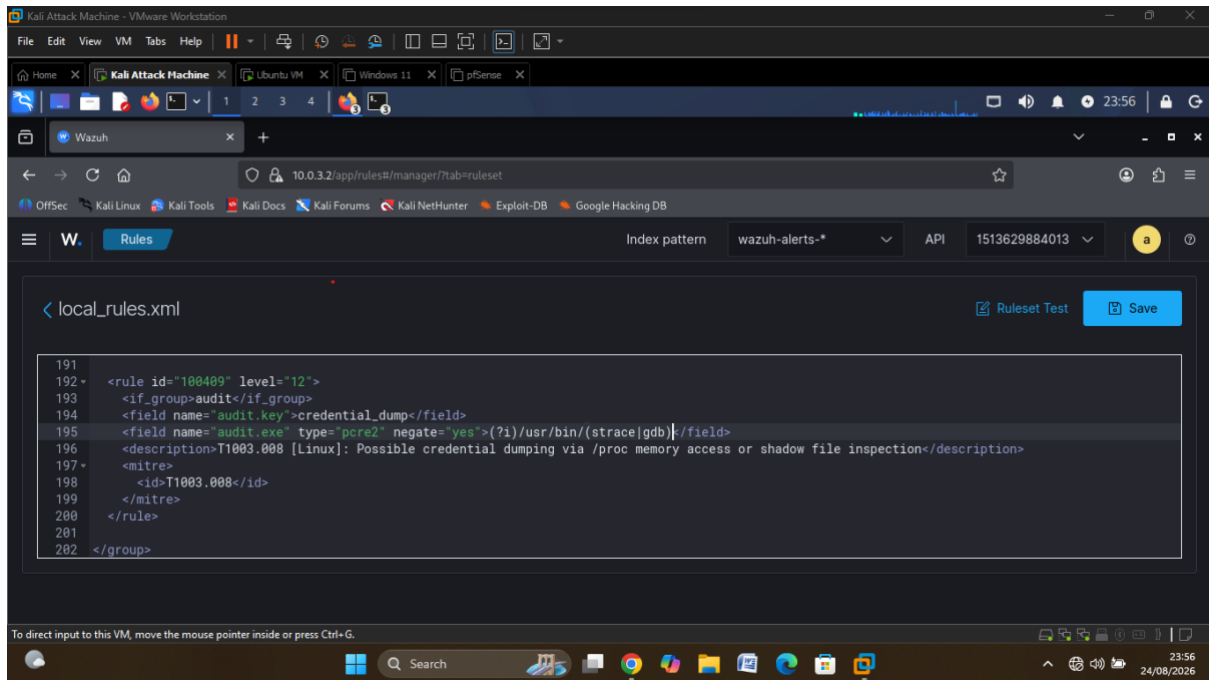
- Tuned Rule 100407.



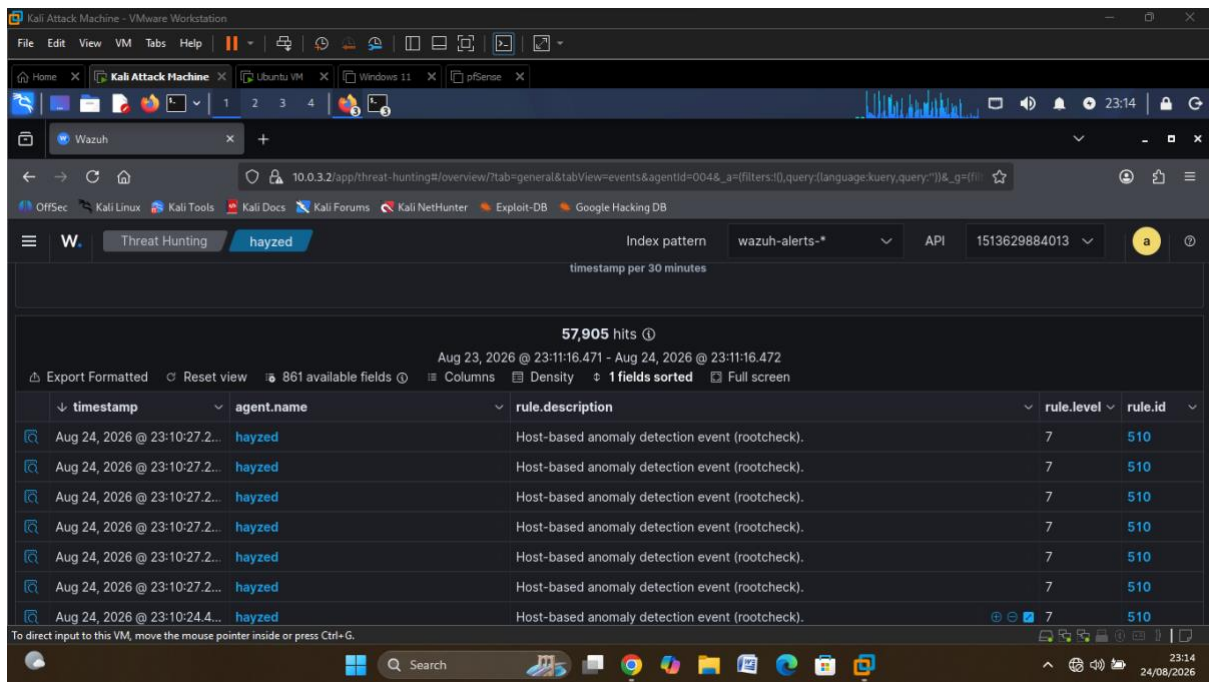
- Tuned Rule 100410.



- Tuned Rule 100409.



- Wazuh event view used to review the resulting event fields after tuning.



3.5 Phase 4 — Triage Workflow

The final phase focused on turning the generated alerts into an actionable SOC workflow.

I created a lightweight triage workflow defining how an analyst should review an alert, determine its severity, investigate the available evidence, and decide whether the alert should be escalated or closed.

The workflow was then applied to nine alerts generated during the project. The alerts included events generated by the custom detection rules as well as system-generated events that required investigation.

For each alert, I documented the relevant evidence, severity, investigation outcome, and final disposition. Two of the nine alerts were identified as false positives, while the remaining seven were generated from the custom detection rules created during the project.

The triage process allowed me to apply the detection results in an analyst-style workflow rather than treating the alert generation itself as the final stage of the project.

4. Findings

The project produced several key findings.

First, the initial log validation identified a gap in the Linux endpoint: Auditd was not installed. This meant that detection rules depending on Auditd could not initially function as intended. Installing and configuring Auditd resolved the issue and allowed the required audit telemetry to reach Wazuh.

Second, the five custom Wazuh detection rules successfully generated alerts when their corresponding activities were simulated. This demonstrated that the rules were operational and that the selected telemetry sources could support the intended detections.

Third, testing exposed the importance of tuning detection logic. Some Auditd-based rules generated significant amounts of repetitive activity. Rule 100409 was the clearest example, producing more than 53,000 Level 12-or-higher alerts within the observed alert set. This showed how an overly broad detection can create alert fatigue even when the underlying detection logic is functioning correctly.

Finally, the triage workflow provided a structured method for handling the alerts produced by the lab. Applying the workflow to nine alerts resulted in documented dispositions and reasoning rather than simply identifying whether an alert had fired.

5. Gaps / Challenges

One of the main challenges encountered during the project was the missing Auditd installation on the Ubuntu endpoint. This was discovered during the log validation phase and had to be resolved before the Auditd-based detections could be tested.

Another challenge was that creating a Wazuh rule alone was not enough for some of the detections. The underlying Auditd configuration also needed to monitor the correct activity and generate events with the appropriate audit keys. This required additional configuration and testing before the Wazuh rules could trigger correctly.

Atomic Red Team also did not provide an exact test for every scenario required by the five detection rules. As a result, some activities had to be manually simulated in the isolated lab environment.

The largest challenge during testing was alert noise. Several Auditd-based rules initially generated repetitive events because their detection criteria matched legitimate or background system activity. This required additional tuning and filtering to make the detections more practical for SOC monitoring.

6. Recommendations

- Continue reviewing detection rules against normal baseline activity as the lab grows.
- Maintain the Auditd configuration alongside the Wazuh rules so that the telemetry required by each detection remains available.
- Periodically review high-volume rules for unnecessary or repetitive alerts.
- Continue using MITRE ATT&CK mappings when developing new detections to maintain a clear relationship between attacker behaviour and detection logic.
- Continue applying the triage workflow to newly generated alerts to improve consistency during investigation.

7. Appendix

APPENDIX A — CUSTOM DETECTION RULES

This appendix contains the custom Wazuh detection rules developed during Phase 2 of the project, including the original detection logic used during development and testing.

1. Rule 100402 — Scheduled Task/Job: Cron

```
<rule id="100402" level="8">

  <if_sid>550,554</if_sid>

  <field name="syscheck.path"
type="pcre2">(?!^)/etc/(cron\.d|cron\. (daily|hourly|weekly|monthly)|crontab)|^/var/spool/cron
/(crontabs/)?.*</field>

  <description>T1053.003 [Linux]: Cron job created or modified for persistence
$(syscheck.path)</description>

  <mitre>

  <id>T1053.003</id>

</mitre>

</rule>
```

2. Rule 100405 — Systemd Service

```
<rule id="100405" level="9">

  <if_sid>550,554</if_sid>

  <field name="syscheck.path"
type="pcre2">(?!^)/(etc|lib|usr/lib)/systemd/system/.*\.service$</field>

  <description>T1543.002 [Linux]: Systemd service unit modified or created
$(syscheck.path)</description>

  <mitre>

  <id>T1543.002</id>

</mitre>

</rule>
```

3. Rule 100407 — Command and Scripting Interpreter: Unix Shell

```
<rule id="100407" level="10">  
  <if_group>audit</if_group>  
  <field name="audit.key">script_interp</field>  
  <field name="audit.exe" type="pcrc2">(?)bin/bash|usr/bin/python3|usr/bin/dash</field>  
  <description>T1059.004 [Linux]: Suspicious interpreter execution  
(bash/python/dash)</description>  
  <mitre>  
    <id>T1059.004</id>  
  </mitre>  
</rule>
```

4. Rule 100409 — OS Credential Dumping

```
<rule id="100409" level="12">  
  <if_group>audit</if_group>  
  <field name="audit.key">credential_dump</field>  
  <description>T1003.008 [Linux]: Possible credential dumping via /proc memory access or  
shadow file inspection</description>  
  <mitre>  
    <id>T1003.008</id>  
  </mitre>  
</rule>
```

5. Rule 100410 — Unsecured Credentials

```
<rule id="100410" level="10">  
  <if_group>audit</if_group>  
  <field name="audit.key">credential_access</field>
```

```
<description>T1552.001 [Linux]: Read access to sensitive credential file
(/etc/shadow)</description>
```

```
<mitre>
```

```
<id>T1552.001</id>
```

```
</mitre>
```

```
</rule>
```

APPENDIX B — TUNED WAZUH DETECTION RULES

The following rules show the tuning changes applied after Phase 3 testing to reduce alert noise while maintaining the intended detection coverage.

1. Rule 100407 — Tuned Command and Scripting Interpreter Detection

```
<rule id="100407" level="10">
```

```
<if_group>audit</if_group>
```

```
<field name="audit.key">script_interp</field>
```

```
<field name="audit.exe"
type="pcre2">(?!i)^(usr/(bin|sbin)/(bash|python3?|dash|perl|php)$|^bin/(bash|dash)</field>
```

```
<field name="audit.tty" type="pcre2">^(pts/d+|tty\d+)</field>
```

```
<description>T1059.004 [Linux]: Suspicious script interpreter execution on terminal
($(audit.tty)) by user ($(audit.exe))</description>
```

```
<mitre>
```

```
<id>T1059.004</id>
```

```
</mitre>
```

```
</rule>
```

Tuning applied:

- Expanded the interpreter matching pattern.
- Added terminal-session filtering using audit.tty.
- Restricted detection to interactive sessions such as pts/X and ttyX.
- Reduced alerts generated by background or non-interactive processes.

2. Rule 100409 — Tuned OS Credential Dumping Detection

```
<rule id="100409" level="12">

  <if_group>audit</if_group>

  <field name="audit.key">credential_dump</field>

  <field name="audit.exe" type="pcr2" negate="yes">(?!)/usr/bin/(strace|gdb)</field>

  <description>T1003.008 [Linux]: Possible credential dumping via /proc memory access or
shadow file inspection</description>

  <mitre>

    <id>T1003.008</id>

  </mitre>

</rule>
```

Tuning applied:

- Added a negative regex condition to exclude designated debugging binaries.
- Reduced repetitive alerts generated by known execution contexts while retaining the credential-dumping detection logic.

3. Rule 100410 — Tuned Unsecured Credentials Detection

```
<rule id="100410" level="10">

  <if_group>audit</if_group>

  <field name="audit.key">credential_access</field>

  <field name="audit.file.name"
type="pcr2">^/etc/(shadow|gshadow|master.passwd)</field>
```

```

    <field name="audit.exe" type="pcre2"
negate="yes">(?!usr/bin/(sudo|su|passwd|chage|gpasswd|login)|usr/sbin/(sshd|cron|unix_ch
kpwd|unattended-upgrade|useradd|usermod))/var/ossec/bin/wazuh-.*</field>

    <description>T1552.001 [Linux]: Unauthorized access attempt to sensitive credential file
($(audit.file.name)) by $(audit.exe)</description>

    <mitre>

    <id>T1552.001</id>

    </mitre>

</rule>

```

Tuning applied:

- Expanded monitoring beyond /etc/shadow to include /etc/gshadow and /etc/master.passwd.
- Added an audit.file.name condition to restrict detection to sensitive credential files.
- Added negative matching to exclude known legitimate administrative tools and system processes.
- Reduced benign credential-access alerts while preserving detection of unauthorized access.

APPENDIX C — AUDITD RULES

The following Auditd rules were created to generate the telemetry required by the custom Wazuh detection rules.

1. Credential Access / Sensitive Credential Files

```
# Rule 100410 / MITRE ATT&CK T1552.001
```

```
-w /etc/shadow -p rwa -k credential_access
```

```
-w /etc/gshadow -p rwa -k credential_access
```

2. Process Memory Dumping

Rule 100409 / MITRE ATT&CK T1003.008

-a always,exit -F arch=b64 -S process_vm_readv,ptrace -k credential_dump

-a always,exit -F arch=b32 -S process_vm_readv,ptrace -k credential_dump

3. Interactive Script Interpreters

Rule 100407 / MITRE ATT&CK T1059.004

-a always,exit -F arch=b64 -S execve -k script_interp

-a always,exit -F arch=b32 -S execve -k script_interp